

Homework 4

PSTAT 131/231 Elliott Liu

Contents

Resampling	1
Required for 231 Students	8

Resampling

For this assignment, we will be working with **two** of our previously used data sets – one for classification and one for regression. For the classification problem, our goal is (once again) to predict which passengers would survive the Titanic shipwreck. For the regression problem, our goal is (also once again) to predict abalone age.

Load the data from `data/titanic.csv` and `data/abalone.csv` into *R* and refresh your memory about the variables they contain using their attached codebooks.

```
library(tidyverse)
library(tidymodels)
library(ggplot2)
library(glmnet)
library(themis)
library(yardstick)
abalone <- read.csv("abalone.csv")
titanic <- read.csv("titanic.csv")
```

Make sure to change `survived` and `pclass` to factors, as before, and make sure to generate the `age` variable as `rings + 1.5`!

```
titanic <- titanic %>% mutate(survived = factor(survived, level = c("Yes", "No")),
                             pclass = factor(pclass))

abalone <- abalone %>% mutate(age = rings + 1.5) %>% select(-rings)
```

Remember that you'll need to set a seed at the beginning of the document to reproduce your results.

```
set.seed(131)
```

Section 1: Regression (abalone age)

Question 1 Follow the instructions from [Homework 2](#) to split the data set, stratifying on the outcome variable, `age`. You can choose the proportions to split the data into. Use *k*-fold cross-validation to create 5 folds from the training set.

Set up the same recipe from [Homework 2](#).

```

abalone_split <- abalone %>% initial_split(strata = age, prop = 0.75)
train <- training(abalone_split)
test <- testing(abalone_split)
folds <- vfold_cv(train, v = 5, strata = age)
abalone_recipe <- recipe(age ~., data = train) %>% step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("type"):shucked_weight +
    longest_shell:diameter +
    shucked_weight:shell_weight) %>%
  step_normalize(all_predictors())

```

Question 2 In your own words, explain what we are doing when we perform k -fold cross-validation:

- What is k -fold cross-validation?

It is a model evaluation method where we divide the training data into k folds, and we train the data k times. Each time we will use $k-1$ folds to train the data, and the remaining 1 fold is used for validation. Every fold could be the validation set once. In the end, we take the average of the k runs to estimate the model's performance.

- Why should we use it, rather than simply comparing our model results on the entire training set?

Evaluating on the entire training set can give overly optimistic results since the model has already seen the data. However, k fold cross validation test the model on unseen data for several times, which can give us a more reliable estimate of the model performance.

- If we split the training set into two and used one of those two splits to evaluate/compare our models, what resampling method would we be using?

The resampling method would be holdout method or validation set approach.

Question 3 Set up workflows for three models:

1. k -nearest neighbors with the `kknn` engine, tuning `neighbors`;
2. linear regression;
3. elastic net **linear** regression, tuning `penalty` and `mixture`.

Use `grid_regular` to set up grids of values for all of the parameters we're tuning. Use values of `neighbors` from 1 to 10, the default values of `penalty`, and values of `mixture` from 0 to 1. Set up 10 levels of each.

```

knn_reg <- nearest_neighbor(neighbors = tune()) %>%
  set_engine("kknn") %>%
  set_mode("regression")

linear_model <- linear_reg() %>%
  set_engine("lm") %>%
  set_mode("regression")

en_model <- linear_reg(mixture = tune(), penalty = tune()) %>%
  set_engine("glmnet") %>%
  set_mode("regression")

```

```
knn_wf <- workflow() %>% add_recipe(abalone_recipe) %>% add_model(knn_reg)
en_wf <- workflow() %>% add_recipe(abalone_recipe) %>% add_model(en_model)
lm_wf <- workflow() %>% add_recipe(abalone_recipe) %>% add_model(linear_model)

knn_grid <- grid_regular(neighbors(range = c(1, 10)), levels = 10)
en_grid <- grid_regular(mixture(range = c(0, 1)), penalty(), levels = 10)
```

How many models total, **across all folds**, will we be fitting to the **abalone data**? To answer, think about how many folds there are, how many combinations of model parameters there are, and how many models you'll fit to each fold.

There are 10 models in knn, 1 model in linear regression, and $10 * 10 = 100$ models in elastic net. Therefore, each fold has $10 + 1 + 100 = 111$ models. Since there are 5 folds in total. Therefore, there are $5 * 111 = 555$ models in total.

Question 4 Fit all the models you created in Question 3 to your folded data.

*Suggest using `tune_grid()`; see the documentation and examples included for help by running `?tune_grid`. You can also see the code in **Lab 4** for help with the tuning process.*

```
rmse_metrics <- metric_set(rmse)

knn_fit <- tune_grid(knn_wf,
  resamples = folds,
  grid = knn_grid,
  metrics = rmse_metrics)

en_fit <- tune_grid(en_wf,
  resamples = folds,
  grid = en_grid,
  metrics = rmse_metrics)

lm_fit <- fit_resamples(lm_wf,
  resamples = folds,
  metrics = rmse_metrics)
```

Question 5 Use `collect_metrics()` to print the mean and standard errors of the performance metric **root mean squared error (RMSE)** for each model across folds.

Decide which of the models has performed the best. Explain how/why you made this decision. Note that each value of the tuning parameter(s) is considered a different model; for instance, KNN with $k = 4$ is one model, KNN with $k = 2$ another.

```
collect_metrics(knn_fit) %>% filter(.metric == "rmse") %>%
  select(neighbors, mean, std_err) %>%
  arrange(mean)
```

```
## # A tibble: 10 x 3
##   neighbors mean std_err
##       <int> <dbl> <dbl>
## 1         10  2.30  0.0306
## 2          9  2.32  0.0312
```

```
## 3      8 2.33 0.0322
## 4      7 2.36 0.0335
## 5      6 2.38 0.0351
## 6      5 2.42 0.0372
## 7      4 2.48 0.0390
## 8      3 2.57 0.0420
## 9      2 2.72 0.0463
## 10     1 2.92 0.0503
```

```
collect_metrics(lm_fit) %>% filter(.metric == "rmse") %>%
  select(mean, std_err) %>%
  arrange(mean)
```

```
## # A tibble: 1 x 2
##   mean std_err
##   <dbl> <dbl>
## 1  2.17  0.0414
```

```
collect_metrics(en_fit) %>% filter(.metric == "rmse") %>%
  select(penalty, mixture, mean, std_err) %>%
  arrange(mean)
```

```
## # A tibble: 100 x 4
##   penalty mixture mean std_err
##   <dbl> <dbl> <dbl> <dbl>
## 1 0.0000000001 0.556 2.17 0.0414
## 2 0.00000000129 0.556 2.17 0.0414
## 3 0.0000000167 0.556 2.17 0.0414
## 4 0.000000215 0.556 2.17 0.0414
## 5 0.00000278 0.556 2.17 0.0414
## 6 0.0000359 0.556 2.17 0.0414
## 7 0.000464 0.556 2.17 0.0414
## 8 0.0000000001 0.444 2.17 0.0414
## 9 0.00000000129 0.444 2.17 0.0414
## 10 0.0000000167 0.444 2.17 0.0414
## # i 90 more rows
```

The linear regression has the lowest mean RMSE, which is 2.17017 across the 5 folds, followed by elastic net linear regression, which has RMSE of 2.170442, followed by knn which has RMSE of 2.304128. Although the difference between linear regression and elastic net is negligible and both models have a very similar performance, linear regression model has the smallest mean RMSE from the result. Also, a lower RMSE means that better predictive accuracy; therefore, we can say that linear regression model performs the best among all models.

Question 6 Use `finalize_workflow()` and `fit()` to fit your chosen model to the entire **training set**.

Lastly, use `augment()` to assess the performance of your chosen model on your **testing set**. Compare your model's **testing** RMSE to its average RMSE across folds.

```
final_model <- lm_wf %>% fit(data = train)
test_pred <- augment(final_model, test)
rmse(test_pred, truth = age, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>         <dbl>
## 1 rmse    standard         2.14
```

The cross validation RMSE for the linear regression model is 2.17017, which is very close to the testing RMSE which is 2.143016. Since the difference is very small, this means that the model generalize well and it does not show any evidence of overfitting or underfitting. Also, since linear regression model has no tuning parameters, the workflow does not require `finalize_workflow()`.

Section 2: Classification (Titanic survival)

Question 7 Follow the instructions from [Homework 3](#) to split the data set, stratifying on the outcome variable, `survived`. You can choose the proportions to split the data into. Use k -fold cross-validation to create 5 folds from the training set.

```
data_split <- initial_split(titanic, prop = 0.75, strata = survived)
train_data <- training(data_split)
test_data <- testing(data_split)

cv_folds <- vfold_cv(train_data, v = 5, strata = survived)
```

Question 8 Set up the same recipe from [Homework 3](#) – but this time, add `step_upsample()` so that there are equal proportions of the Yes and No levels (you’ll need to specify the appropriate function arguments).

Note: See Lab 5 for code/tips on handling imbalanced outcomes.

```
titanic_recipe <- recipe(survived~pclass + sex + age + sib_sp + parch + fare, data = train_data) %>%
  step_impute_linear(age, impute_with = imp_vars(all_predictors())) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(term = ~starts_with("sex"): age + age:fare) %>%
  step_upsample(survived, over_ratio = 1)
```

Question 9 Set up workflows for three models:

1. k -nearest neighbors with the `kkn` engine, tuning `neighbors`;
2. logistic regression;
3. elastic net **logistic** regression, tuning `penalty` and `mixture`.

Set up the grids, etc. the same way you did in [Question 3](#). Note that you can use the same grids of parameter values without having to recreate them.

```
knn_cla <- nearest_neighbor(neighbors = tune()) %>%
  set_engine("kkn") %>%
  set_mode("classification")

log_cla <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

en_cla <- logistic_reg(mixture = tune(), penalty = tune()) %>%
```

```

set_engine("glmnet") %>%
set_mode("classification")

knn_wf_c <- workflow() %>%
  add_recipe(titanic_recipe) %>%
  add_model(knn_cla)

log_wf_c <- workflow() %>%
  add_recipe(titanic_recipe) %>%
  add_model(log_cla)

en_wf_c <- workflow() %>%
  add_recipe(titanic_recipe) %>%
  add_model(en_cla)

```

Question 10 Fit all the models you created in Question 9 to your folded data.

```

knn_res <- tune_grid(knn_wf_c,
  resamples = cv_folds,
  grid = knn_grid,
  metrics = metric_set(roc_auc, accuracy))

log_res <- fit_resamples(log_wf_c,
  resamples = cv_folds,
  metrics = metric_set(roc_auc, accuracy))

en_res <- tune_grid(en_wf_c,
  resamples = cv_folds,
  grid = en_grid,
  metrics = metric_set(roc_auc, accuracy))

```

Question 11 Use `collect_metrics()` to print the mean and standard errors of the performance metric *area under the ROC curve* for each model across folds.

Decide which of the models has performed the best. Explain how/why you made this decision.

```

collect_metrics(knn_res) %>% filter(.metric == "roc_auc") %>%
  select(neighbors, mean, std_err) %>%
  arrange(desc(mean))

```

```

## # A tibble: 10 x 3
##   neighbors mean std_err
##   <int> <dbl> <dbl>
## 1      10 0.840 0.0116
## 2       9 0.835 0.0134
## 3       8 0.833 0.0116
## 4       7 0.831 0.0127
## 5       6 0.829 0.0128
## 6       5 0.824 0.0154
## 7       4 0.818 0.0147
## 8       3 0.810 0.0193
## 9       2 0.783 0.0192

```

```
## 10          1 0.744  0.0173
```

```
collect_metrics(log_res) %>% filter(.metric == "roc_auc") %>%  
  select(mean, std_err) %>%  
  arrange(desc(mean))
```

```
## # A tibble: 1 x 2  
##   mean std_err  
##   <dbl>   <dbl>  
## 1 0.853  0.0142
```

```
collect_metrics(en_res) %>% filter(.metric == "roc_auc") %>%  
  select(penalty, mixture, mean, std_err) %>%  
  arrange(desc(mean))
```

```
## # A tibble: 100 x 4  
##   penalty mixture mean std_err  
##   <dbl>   <dbl> <dbl>   <dbl>  
## 1 0.00599      0.111 0.854  0.0155  
## 2 0.000464     0.222 0.853  0.0146  
## 3 0.000464     0.333 0.853  0.0146  
## 4 0.00599      0.222 0.853  0.0153  
## 5 0.00599      0.333 0.853  0.0154  
## 6 0.000464     0.556 0.853  0.0145  
## 7 0.000464     0.444 0.853  0.0145  
## 8 0.000464     0.778 0.853  0.0143  
## 9 0.000464     0.667 0.853  0.0145  
## 10 0.0000000001 0.444 0.853  0.0146  
## # i 90 more rows
```

Based on the mean ROC AUC across folds, the elastic net logistic regression has the highest performance (with $AUC \approx 0.854$) followed by logistic regression model which has $AUC \approx 0.853$, followed by KNN (0.840). Although there is a little difference between the result of the elastic net logistic regression model and logistic regression model, it is too small and is negligible. Therefore, both models have a very similar performance. Since elastic net logistic regression has a slightly higher ROC_AUC value; therefore, we can say elastic net logistic regression model performs the best.

Question 12 Use `finalize_workflow()` and `fit()` to fit your chosen model to the entire **training set**.

Lastly, use `augment()` to assess the performance of your chosen model on your **testing set**. Compare your model's **testing** ROC AUC to its average ROC AUC across folds.

```
best_params <- select_best(en_res, metric = "roc_auc")  
final_wf <- finalize_workflow(en_wf_c, best_params)  
  
final_fit <- final_wf %>%  
  fit(data = train_data)  
  
test_prediction <- augment(final_fit, test_data)  
roc_auc(test_prediction, truth = survived, .pred_Yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>         <dbl>
## 1 roc_auc binary         0.861
```

The elastic net logistic regression model has a testing ROC AUC of 0.861, which is slightly greater than the cross validation average ROC AUC value which is 0.854. Since the difference is very small, this means that the model generalize well and does not show evidence of overfitting or underfitting.

Required for 231 Students

Consider the following intercept-only model, with $\epsilon \sim N(0, \sigma^2)$:

$$Y = \beta + \epsilon$$

where β is the parameter that we want to estimate. Suppose that we have n observations of the response, i.e. $y_1, \dots, y_n$, with uncorrelated errors.

Question 13

Derive the least-squares estimate of β .

Question 14

Suppose that we perform leave-one-out cross-validation (LOOCV). Recall that, in LOOCV, we divide the data into n folds.

Derive the covariance between $\hat{\beta}^{(1)}$, or the least-squares estimator of β that we obtain by taking the first fold as a training set, and $\hat{\beta}^{(2)}$, the least-squares estimator of β that we obtain by taking the second fold as a training set?